



Αποδομώντας το 3D Gaussian Splatting: Από τα Gaussians στην Εικόνα, μόνο με PyTorch

Μια υλοποίηση από το μηδέν, χωρίς custom CUDA kernels, που αποκαλύπτει τον πυρήνα του αλγορίθμου rendering.

Η υπόσχεση: Φωτορεαλισμός με ελάχιστη πολυπλοκότητα

Στόχος: Η πλήρης υλοποίηση της διαδικασίας rendering (splatting) του 3D Gaussian Splatting.

Περιορισμοί: Αποκλειστικά με PyTorch. Χωρίς C++ ή custom CUDA.

Αποτέλεσμα: Η ποιότητα εικόνας είναι εφάμιλλη με την επίσημη υλοποίηση της έρευνας, αν και ελαφρώς πιο αργή λόγω PyTorch.



NumPy

PiL

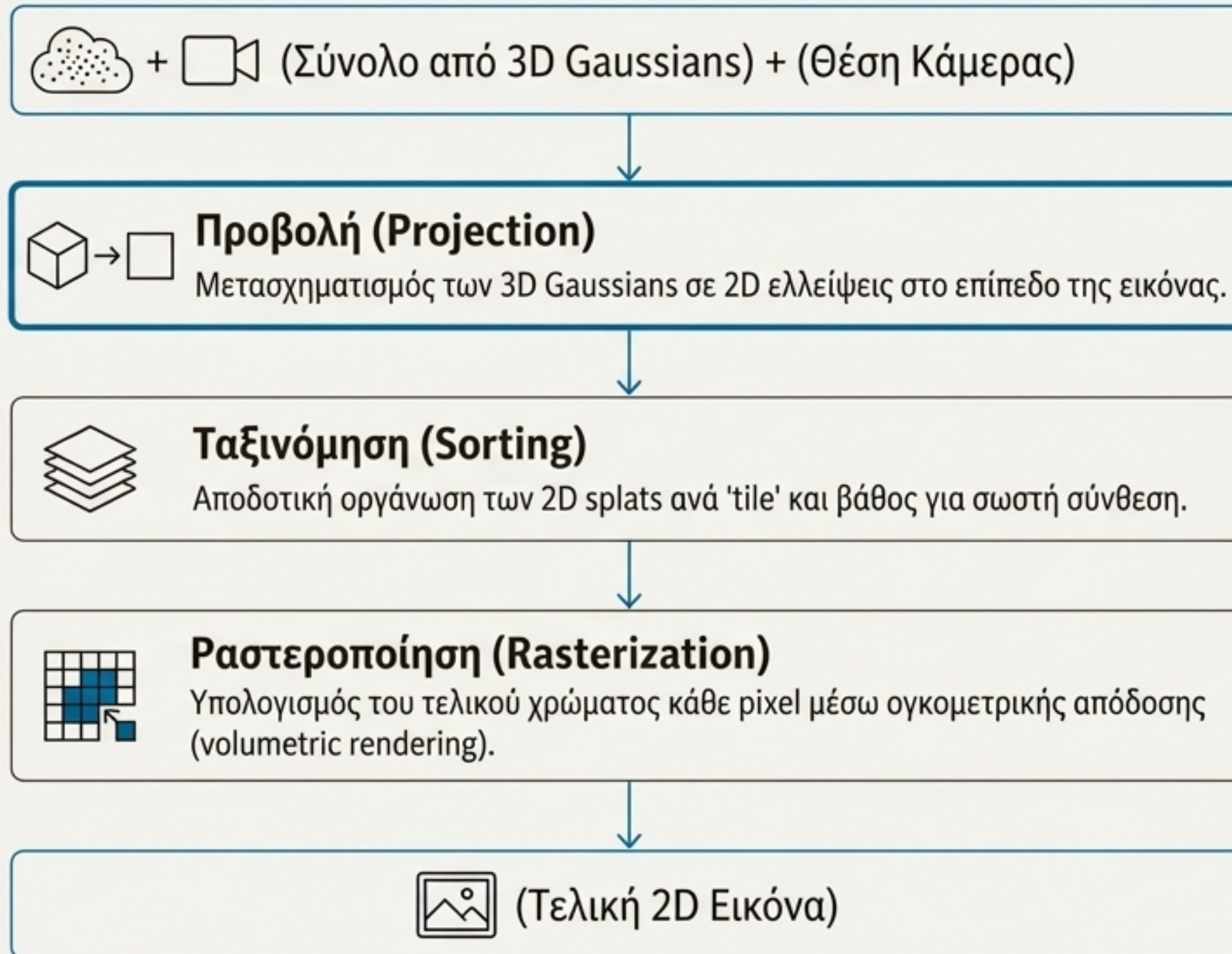


Επίσημη Υλοποίηση
(Paper)



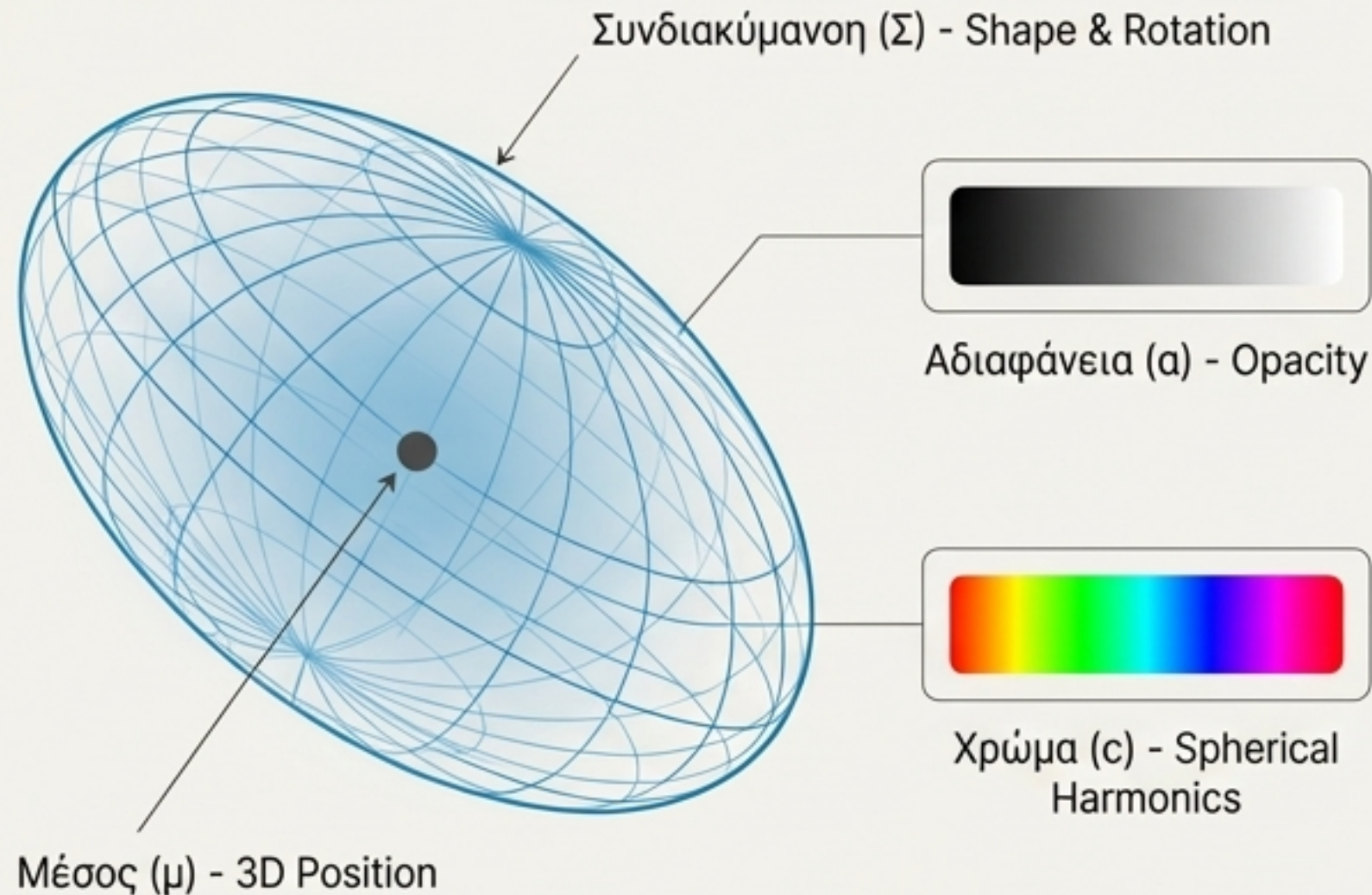
Η Υλοποίησή μας
(PyTorch)

Ο αγωγός rendering: Μια επισκόπηση της διαδικασίας



Θα εξετάσουμε κάθε βήμα αυτής της διαδικασίας παλιωεαίλ.

Η ανατομία ενός 3D Gaussian



Μέσος (Mean): Η θέση του Gaussian στον 3D χώρο.

Συνδιακύμανση (Covariance): Ένας πίνακας 3×3 που ορίζει το σχήμα και τον προσανατολισμό. Για να διασφαλιστεί ότι είναι έγκυρος, δεν τον μαθαίνουμε απευθείας. Αντ' αυτού, τον κατασκευάζουμε από:

- **Κλίμακα (Scale):** Μαθαίνεται ως log-scale για να επιτρέπονται οποιεσδήποτε τιμές.
- **Περιστροφή (Rotation):** Μαθαίνεται ως τετρανύο (quaternion) για σταθερότητα.

Αδιαφάνεια (Opacity): Μια τιμή που μαθαίνεται ανά Gaussian.

Χρώμα (Color): Όχι μια σταθερή τιμή RGB, αλλά συντελεστές Σφαιρικών Αρμονικών (Spherical Harmonics) για χρώμα που εξαρτάται από τη γωνία θέασης.

Το χρώμα εξαρτάται από τη γωνία θέασης: Ο ρόλος των Σφαιρικών Αρμονικών

- Το χρώμα κάθε Gaussian δεν είναι σταθερό. Αλλάζει ανάλογα με την κατεύθυνση από την οποία το παρατηρούμε (**viewing direction**).
- Για να το μοντελοποιήσουμε αυτό, αποθηκεύουμε συντελεστές **Σφαιρικών Αρμονικών (SH)** βαθμού 3.
- Αυτό σημαίνει 16 συντελεστές για κάθε κανάλι χρώματος (R, G, B).
- Κατά το rendering, η συνάρτηση **evaluate_sh** λαμβάνει την κατεύθυνση θέασης (ένα διάνυσμα x, y, z) και τους αποθηκευμένους συντελεστές για να υπολογίσει το τελικό χρώμα.

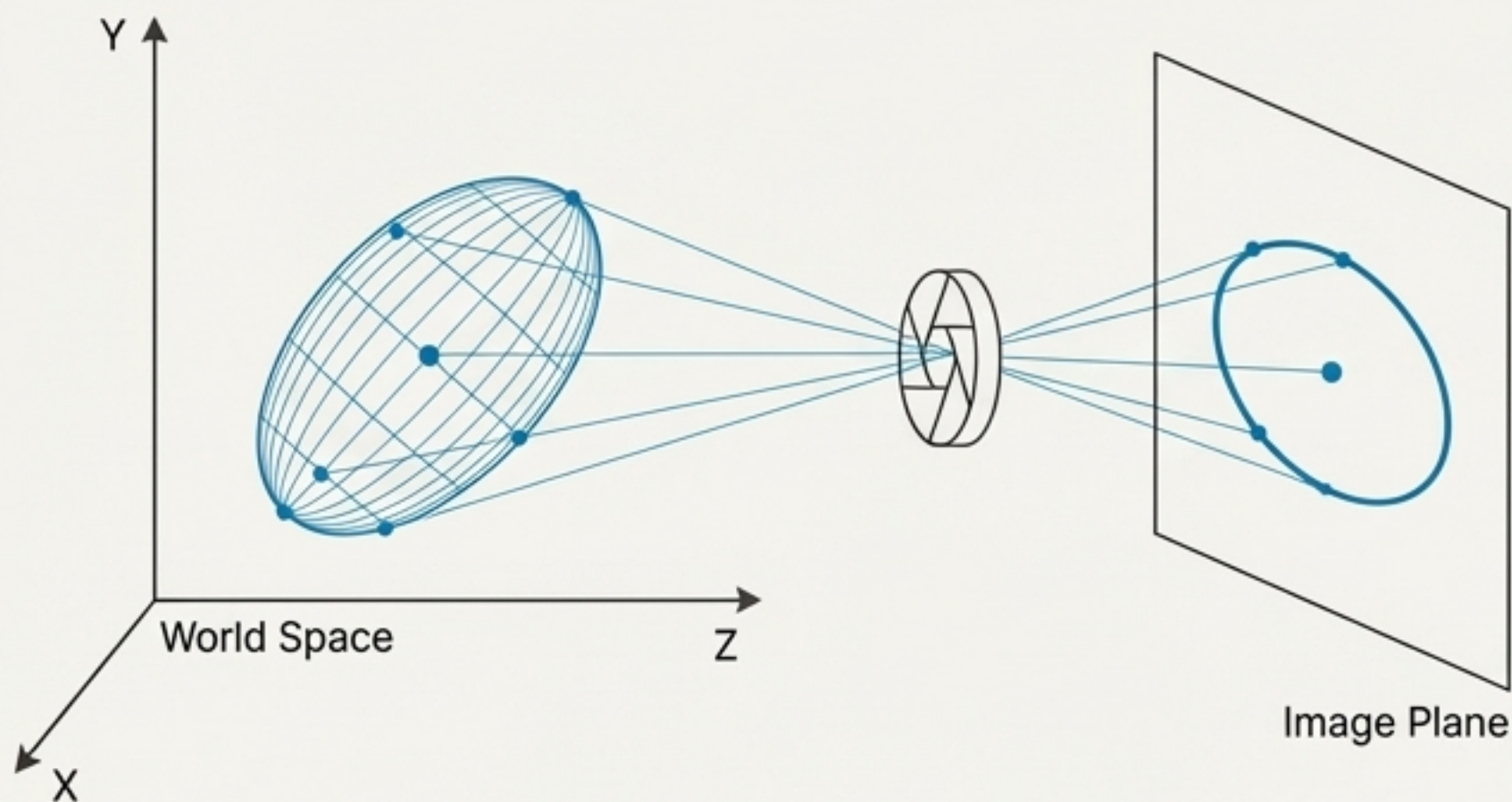


Βήμα 1: Προβολή των Gaussians από τον 3D χώρο στο 2D επίπεδο

Προβολή
(Projection)

Ταξινόμηση
(Sorting)

Ραστεροποίηση
(Rasterization)



Για να ζωγραφίσουμε τα Gaussians, πρέπει πρώτα να τα “προβάλουμε” στην οθόνη. Αυτή η διαδικασία έχει δύο μέρη:

1. **Προβολή του Κέντρου**: Μετατρέπουμε τη 3D θέση (μέσο) του Gaussian σε 2D συντεταγμένες pixel χρησιμοποιώντας την κλασική προοπτική προβολή (**perspective projection**).
2. **Προβολή της Συνδιακύμανσης**: Μετατρέπουμε τον 3D πίνακα συνδιακύμανσης (Σ) σε έναν 2D πίνακα (Σ'), ο οποίος περιγράφει το σχήμα της έλλειψης στην 2D εικόνα.

Ο μετασχηματισμός της συνδιακύμανσης

Η προβολή της συνδιακύμανσης από 3D σε 2D δεν είναι τετριμμένη. Υλοποιεί την Εξίσωση 5 από την έρευνα:

$$\Sigma' = \mathbf{J} \cdot \mathbf{W} \cdot \Sigma \cdot \mathbf{W}^T \cdot \mathbf{J}^T$$

Ο προκύπτων 2x2 πίνακας συνδιακύμανσης στο επίπεδο της εικόνας.

Ο Ιακωβιανός (Jacobian) πίνακας της συνάρτησης προοπτικής προβολής. Αποτυπώνει πώς μια μικρή αλλαγή στον 3D χώρο της κάμερας επηρεάζει τις 2D συντεταγμένες της εικόνας.

Ο 3x4 πίνακας μετασχηματισμού από world space σε camera space.

Ο αρχικός 3x3 πίνακας συνδιακύμανσης στον παγκόσμιο χώρο (world space).

Αυτή η εξίσωση είναι ο μαθηματικός πυρήνας που μετατρέπει ένα 3D ελλειψοειδές στο 2D 'splat' του.

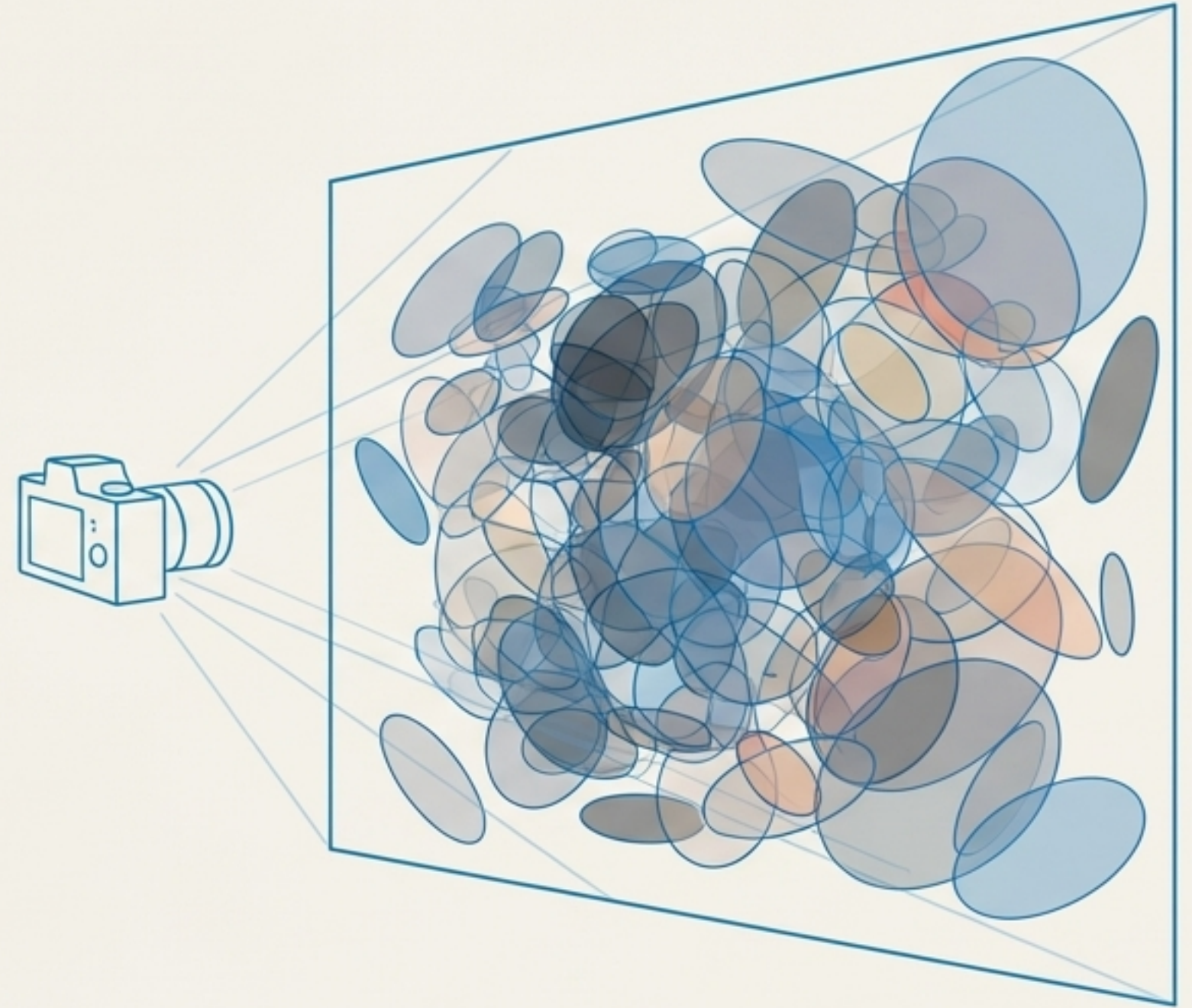
Η πρόκληση της απόδοσης: Εκατομμύρια splats χρειάζονται ταξινόμηση

Έχουμε πλέον **εκατομμύρια 2D splats** στο επίπεδο της εικόνας.

Για να συνθέσουμε σωστά την τελική εικόνα (alpha compositing), πρέπει να τα ζωγραφίσουμε με τη σωστή σειρά: από πίσω προς τα εμπρός (back to front).

Μια 'αφελής' προσέγγιση θα ήταν να ταξινομήσουμε όλα τα splats με βάση το βάθος τους.

Αυτό είναι υπολογιστικά **ασύμφορο** και θα αποτελούσε τεράστιο εμπόδιο στην απόδοση σε πραγματικό χρόνο.



Η λύση, Μέρος 1ο: Παραλληλοποίηση μέσω 'Tiling'

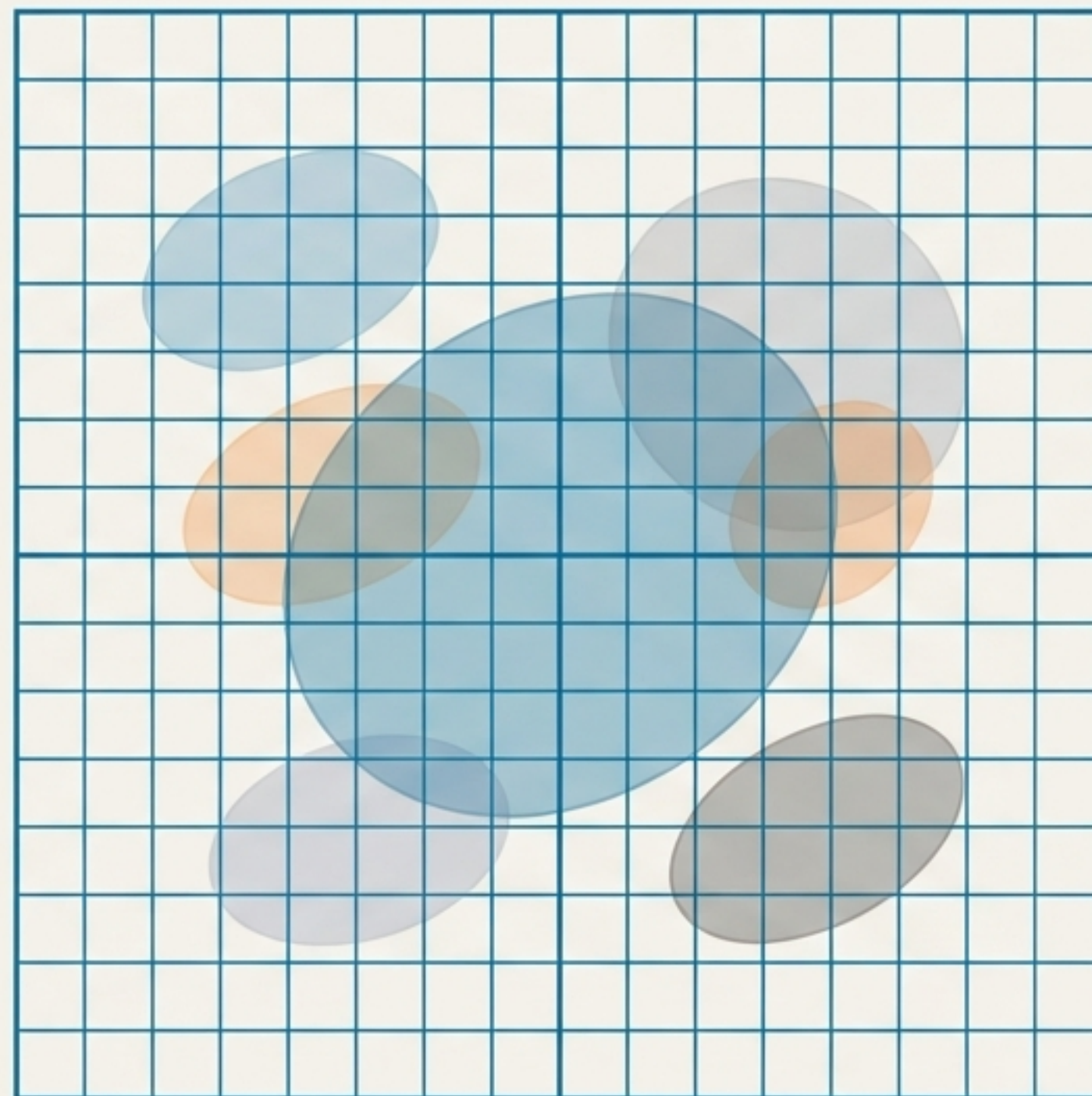
Η βασική ιδέα είναι **‘διαίρει και βασίλευε’**. Αντί να επεξεργαζόμαστε ολόκληρη την εικόνα ταυτόχρονα, τη χωρίζουμε σε ένα πλέγμα από ανεξάρτητα **‘tiles’** (π.χ., 16x16 pixels).

Η επεξεργασία **παραλληλοποιείται** ανά tile, όχι ανά Gaussian.

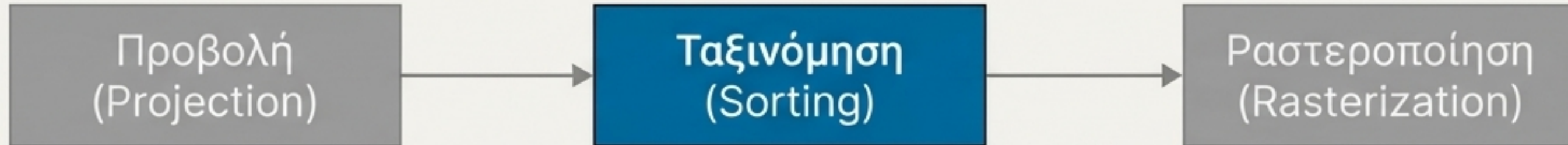
Για κάθε Gaussian, προσδιορίζουμε ποια tiles επικαλύπτει.

Η τελική εικόνα συντίθεται επεξεργάζόμενοι κάθε tile ξεχωριστά.

(Αναφορά: Αυτή η στρατηγική περιγράφεται λεπτομερώς στην έρευνα Pulsar, έναν σημαντικό πρόδρομο του Gaussian Splatting.)



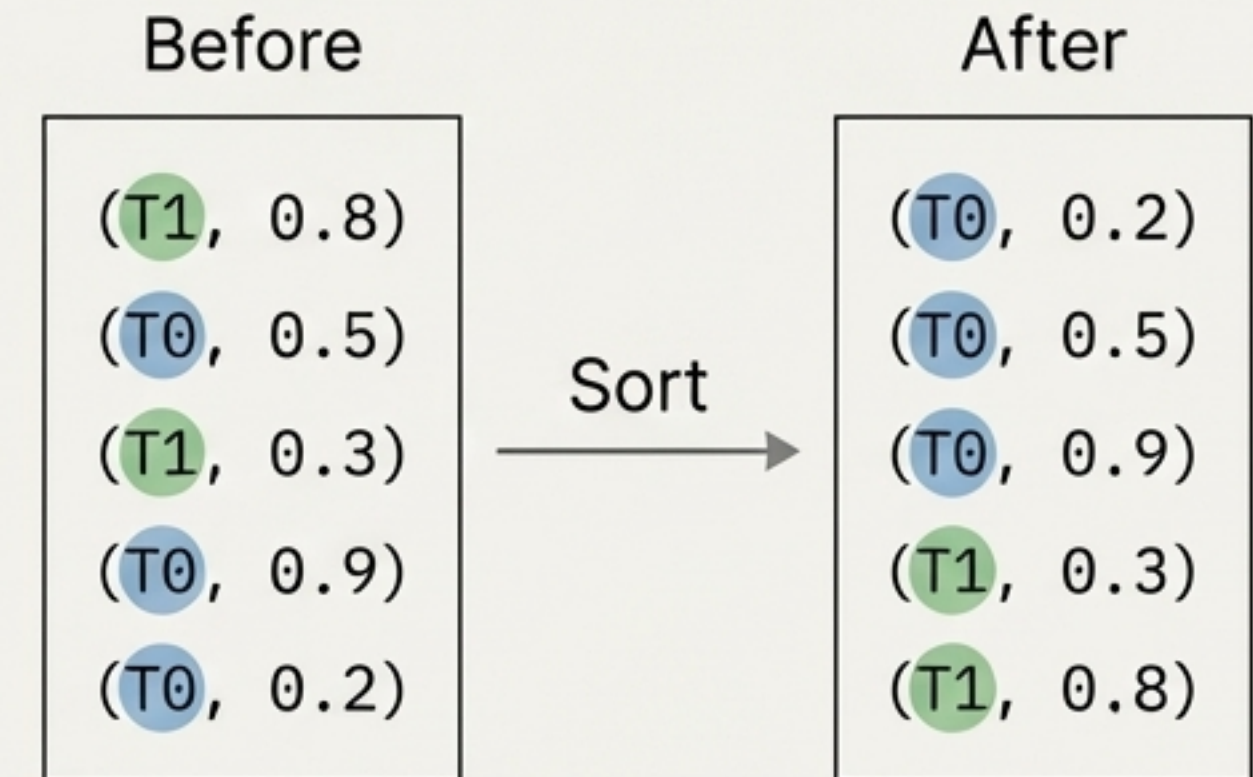
Βήμα 2: Η στρατηγική ταξινόμησης - Λεξικογραφική ταξινόμηση



Για να επεξεργαστούμε τα tiles αποδοτικά, χρειαζόμαστε μια συγκεκριμένη σειρά. Χρησιμοποιούμε λεξικογραφική ταξινόμηση (lexicographical sort) με δύο κλειδιά:

1. **Κύριο Κλειδί: Tile ID.** Όλα τα Gaussians που ανήκουν στο ίδιο tile ομαδοποιούνται.
2. **Δευτερεύον Κλειδί: Βάθος (Depth).** Μέσα σε κάθε ομάδα *tile*, τα Gaussians ταξινομούνται από πίσω προς τα εμπρός.

Αποτέλεσμα: Μια ενιαία, ταξινομημένη λίστα όπου μπορούμε να επεξεργαστούμε διαδοχικά όλα τα Gaussians για το Tile 0, μετά για το Tile 1, κ.ο.κ., με τη σωστή σειρά βάθους.

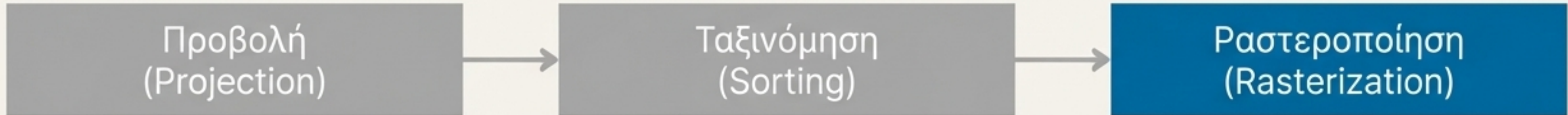


Υλοποιώντας τη λεξικογραφική ταξινόμηση στο PyTorch

Το PyTorch δεν διαθέτει ενσωματωμένη συνάρτηση για λεξικογραφική ταξινόμηση. Το τέχνασμα είναι να συνδυάσουμε τα δύο κλειδιά (**Tile ID**, **Βάθος**) σε ένα ενιαίο, 64-bit ακέραιο κλειδί. Η εξίσωση που χρησιμοποιείται: $\text{combined_key} = \text{tile_id} * M + \text{depth}$ (όπου M είναι μια μεγάλη σταθερά, όπως το μέγιστο βάθος). Στη συνέχεια, εκτελούμε μια τυπική ταξινόμηση σε αυτό το **combined_key**. Αυτό επιτυγχάνει το επιθυμητό αποτέλεσμα: οι τιμές με μικρότερο **tile_id** θα έρθουν πρώτες, και για ίδιο **tile_id**, αυτές με μικρότερο **depth** θα προηγηθούν.

```
# Assume depths are quantized to integers
# M is a large number, e.g., max_depth
M = 2**32
combined_keys = tile_ids * M + depth_keys
sorted_indices = torch.argsort(combined_keys)
```

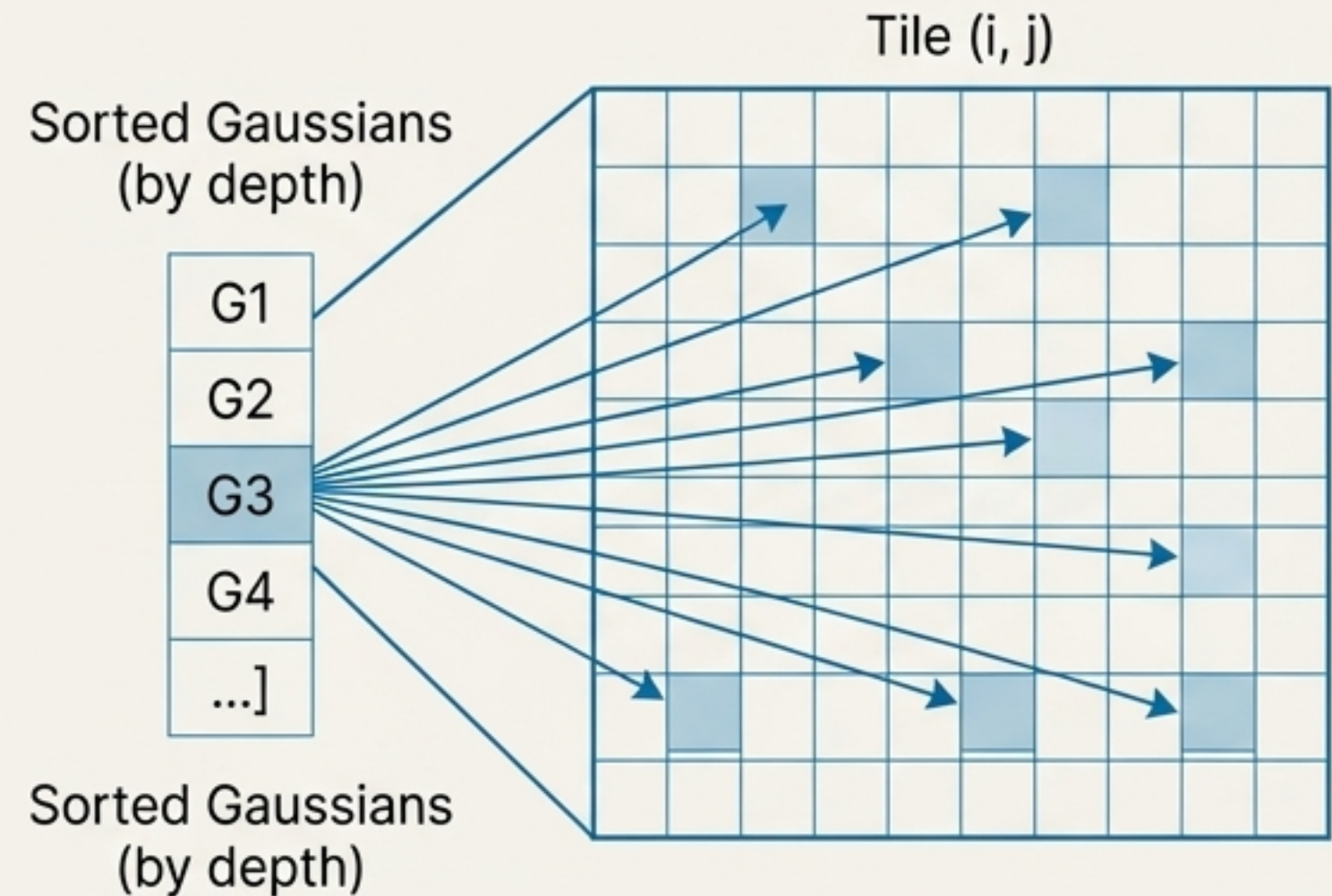

Βήμα 3: Ραστεροποίηση & Ογκομετρική Απόδοση



Με τα Gaussians ταξινομημένα, είμαστε έτοιμοι να “ζωγραφίσουμε” την εικόνα, tile προς tile.

Η διαδικασία:

1. Επαναλαμβάνουμε για κάθε tile στην οθόνη.
2. Για κάθε pixel μέσα στο τρέχον tile:
 3. Επαναλαμβάνουμε μέσα από τη (ταξινομημένη) λίστα των Gaussians που αντιστοιχούν σε αυτό το tile.
4. Εφαρμόζουμε τον τύπο της ογκομετρικής απόδοσης (volumetric rendering) για να συνδυάσουμε τα χρώματα και τις αδιαφάνειες, από πίσω προς τα εμπρός.



Ο τύπος της ογκομετρικής απόδοσης

$$\mathbf{C} = \sum \mathbf{T}_i \cdot \alpha_i \cdot \mathbf{c}_i$$

$\mathbf{C}$: Το τελικό χρώμα του pixel.

$\mathbf{c}_i$: Το (εξαρτώμενο από τη γωνία θέασης) χρώμα του i -οστού Gaussian.

α_i (**Alpha**): Η "συμβολή" του i -οστού Gaussian. Υπολογίζεται ως το γινόμενο της μαθημένης αδιαφάνειάς του και της τιμής της 2D Gaussian PDF στο κέντρο του pixel.

$\alpha_i = \text{opacity}_i \cdot \text{PDF}(\text{pixel_coord})$.

$\mathbf{T}_i$ (**Transmittance**): Η 'διαπερατότητα' του φωτός μέχρι το i -οστό Gaussian. Είναι το γινόμενο όλων των $(1 - \alpha_j)$ για τα Gaussians j που βρίσκονται μπροστά από το i .

Αν και η αναπαράσταση της σκηνής είναι διαφορετική, ο τελικός συνδυασμός χρωμάτων ακολουθεί τις ίδιες αρχές με το NeRF.

Συνδέοντας τα κομμάτια: Η δομή του κεντρικού script

```
# 1. Φόρτωση δεδομένων
gaussians = load_gaussians("kitchen_7k.ply")
cameras = load_camera_path(...)

# 2. Υπολογισμός πινάκων συνδιακύμανσης (ΜΟΝΟ ΜΙΑ ΦΟΡΑ)
sigma = build_sigma_from_params(gaussians)

# 3. Κεντρικός βρόχος απόδοσης (rendering loop)
for cam_matrix in cameras:
    # Υπολογισμός χρώματος για τη νέα γωνία θέασης
    colors = evaluate_sh(gaussians, cam_matrix)

    # Κλήση της κύριας συνάρτησης rendering
    image = render(gaussians, colors, sigma, cam_matrix)

    # Αποθήκευση εικόνας
    save_image(image)
```

Δείχνει τη ροή: η ακριβή γεωμετρία (sigma) υπολογίζεται μία φορά, ενώ η εμφάνιση (χρώματα) ενημερώνεται δυναμικά σε κάθε καρέ.

Η δύναμη της αλγοριθμικής κομψότητας

Διανύσαμε το μονοπάτι από εκατομμύρια αφηρημένες **3D Gaussians** σε μια φωτορεαλιστική, κινούμενη εικόνα.

Το πετύχαμε αυτό με λίγες εκατοντάδες γραμμές κώδικα, αποκλειστικά σε PyTorch.

Η "μαγεία" του **3D Gaussian Splatting** δεν βρίσκεται σε custom CUDA kernels, αλλά σε έναν **έξυπνο, παραλληλοποιήσιμο αλγόριθμο**:

- Η προβολή της **συνδιακύμανσης**.
- Η ραστεροποίηση βασισμένη σε tiles.
- Η αποδοτική **λεξικογραφική ταξινόμηση**.

